# TESSERA

Identity and authority for AI agents

Working Draft v0.7

Guglielmo Reggio

April 2026

---

*The Roman* tessera hospitalis *was a token split between two strangers. Reuniting the pieces proved a relationship of trust.*

# Contents

**Abstract**

AI agents can now execute shell commands, modify files, send messages, call APIs, and trigger real-world side effects. The main security problem is no longer model output quality alone. It is ambient authority: agents routinely inherit broad, implicit permissions that are hard to scope, hard to revoke, and difficult to audit.

Tessera is a capability-based security layer for agent actions. At the product layer, Tessera provides identity and authority for AI agents. At the technical layer, it issues scoped capabilities, enforces them at runtime, revokes them quickly, and records decision traces for operators and platforms.

Tessera starts with a narrow wedge: runtime-native enforcement inside real agent runtimes such as OpenClaw. The goal is not to solve every aspect of AI safety or semantic policy. The goal is to make sensitive agent actions require explicit, revocable, runtime-enforced authority.

The long-term opportunity is larger. If platforms, runtimes, and operators adopt common authority checks, agents can become easier to trust, easier to constrain, and easier to audit. But that future depends on practical adoption, not protocol elegance. Tessera therefore starts with the most concrete problem first: capability-based security for agent actions.

## 1. Executive Summary

AI agents are moving from suggestion to execution. They can already send messages, edit code, access tools, and operate software on behalf of users and organisations. Most of these systems still rely on ambient authority: if an agent has access to a token, tool, or runtime, it can often act with far broader power than was intentionally delegated.

Tessera addresses that problem by introducing scoped, revocable, runtime-enforced authority for agent actions.

The core claim of this paper is simple:

> agent actions should require explicit capabilities, and those capabilities should be scoped, revocable, enforced at runtime, and auditable.

This makes Tessera closer to OAuth for agent actions than to a generic identity layer. The point is not merely to name an agent. The point is to control what that agent can do, when it can do it, and how that authority can be stopped.

Today, Tessera's most credible wedge is runtime-native enforcement in real agent runtimes such as OpenClaw. That wedge is intentionally narrow, but it is real: scoped capabilities, runtime hooks, revocation, and audit evidence inside an actual execution path.

# 2. The Problem

## 2.1 Agents inherit too much authority

Modern agent systems are increasingly able to:

- execute shell commands,
- edit code and files,
- send outbound messages,
- trigger third-party APIs,
- spend money or consume quota,
- and coordinate across tools and services.

The default security model is often weak. If the runtime can access a tool, the agent can usually attempt to use it. If the operator has already authenticated a service, the agent may inherit those credentials. If there is a manual approval step, it is often either too coarse, too frequent, or too easy to bypass by moving the action elsewhere in the stack.

## 2.2 Ambient authority is the real failure mode

The central failure mode is ambient authority. An agent does not prove explicit authority for each sensitive class of action. It simply acts with whatever authority the surrounding runtime, tool, token, or session already exposes.

That creates predictable problems:

- permissions are broader than intended,
- delegation is implicit rather than explicit,
- revocation is slow or nonexistent,
- audit trails are incomplete,
- and operators cannot clearly answer why an action was allowed or denied.

## 2.3 Existing patterns are insufficient

Most current approaches fall into one of four categories:

1. **Direct tool access.** If the agent has the tool, it can try to use it.
2. **Approval spam.** The human is asked repeatedly, which undermines autonomy and encourages approval fatigue.
3. **Framework-specific permissions.** Each runtime invents its own policy surface, often with weak portability or poor revocation semantics.
4. **Proxy-only controls.** Enforcement happens outside the runtime hot path and can drift from what the agent is actually doing.

None of these gives a clean answer to the core operator questions:

- what was this agent allowed to do,
- under whose authority,
- with what limits,
- and how do we stop it now?

## 3. Thesis

The thesis of Tessera is that agent actions should require explicit capabilities.

Those capabilities should be:

- **scoped** to specific action classes and limits,
- **runtime-enforced** at the point where the action actually executes,
- **revocable** quickly enough to function as a real kill switch,
- and **auditable** so operators can reconstruct what was requested, allowed, denied, granted, or revoked.

This is the deepest technical framing of Tessera:

> Tessera is a capability-based security layer for agent actions.

Identity still matters, but it is not the deepest layer. Identity tells a system which principal or agent is involved. Capabilities determine what that agent is allowed to do.

## 4. What Tessera Is

Tessera operates at three levels of framing.

### 4.1 Product framing

Tessera provides identity and authority for AI agents.

### 4.2 Technical framing

Tessera provides capability-based security for agent actions.

### 4.3 Operational framing

Tessera gives agents scoped, revocable, runtime-enforced authority with audit trails.

In practical terms, Tessera is a control layer that sits in or near the runtime path where agent actions execute. It checks whether an agent has a valid capability for the requested action, whether that capability remains in force, and whether the action should be allowed or denied.

The four core pillars are:

1. **Scoped capabilities**
2. **Runtime enforcement**
3. **Instant revocation**
4. **Auditability**

## 5. What Tessera Is Not

Tessera is intentionally narrower than several adjacent narratives.

Tessera is not:

- a blockchain system,
- a decentralised ledger project,
- an antivirus product,
- a general semantic moderation engine,
- a full AI safety solution,
- or a universal ideological identity framework.

Tessera does not currently judge whether the content of an action is morally acceptable, manipulative, fraudulent, or otherwise semantically harmful. Its present job is narrower and more concrete: to enforce authority boundaries around action types, scope, expiry, limits, revocation, and operator visibility.

## 6. System Model

The Tessera system model has six core actors and stores.

### 6.1 Principal

The principal is the human or organisation on whose behalf authority is delegated.

### 6.2 Agent

The agent is the runtime subject requesting an action. In practice this may be represented by an agent id, key, session context, or runtime-specific identity.

### 6.3 Issuer

The issuer mints capabilities or credentials that bind delegated authority to an agent within a defined scope.

### 6.4 Verifier / guard

The verifier is the runtime hook or guard layer that evaluates whether a requested action is authorised.

### 6.5 Revocation registry

The revocation registry records whether a capability or delegation instance has been invalidated.

### 6.6 Audit log

The audit log records the decision trace: what was requested, what was presented, what was allowed or denied, when authority changed, and why the result occurred.

In a minimal deployment, these components may all be local. In a larger deployment, the issuer, revocation registry, and audit plane may be managed services.

## 7. The Capability Model

Tessera capabilities define what an agent is allowed to do. A capability may include:

- action classes such as `exec.shell`, `message.send`, or `code.write`,
- quantitative limits such as spend caps or recipient limits,
- time bounds such as issued-at and expiry,
- runtime or agent binding,
- and a unique delegation id for revocation and audit.

Illustrative shape:

```
{
  "type": "TesseraCapability",
  "agentId": "main",
  "delegationId": "a4c9b7d8-...",
  "scope": {
    "actions": ["message.send"],
    "maxRecipients": 1
  },
```

```
  "issuedAt": 1775220000,
  "expiresAt": 1775223600,
  "issuer": "tessera-demo",
  "signature": "..."
}
```

The point of the capability is not to describe everything an agent is. The point is to constrain what it may do.

## 8. Enforcement Boundary

The strongest Tessera integrations sit at real runtime boundaries, not just at dashboards or proxies.

Examples of enforcement boundaries include:

- tool invocation hooks such as `before_tool_call`,
- outbound message hooks such as `message_sending`,
- gateway-side action handlers,
- and server-side tool execution boundaries.

This matters because enforcement is hardest to bypass when it sits in the execution path itself.

### 8.1 Current limitation: semantic enforcement is not solved

Tessera does not yet perform semantic enforcement.

Today, Tessera can enforce:

- whether an action type is permitted,
- whether the requested action is in scope,
- whether a delegation has expired,
- whether a delegation has been revoked,
- and whether quantitative bounds are exceeded.

It cannot yet reliably determine whether an allowed payload is benign or harmful.

For example:

- if an agent has permission to send email or send a message, Tessera can enforce that the send action is permitted within bounds,
- but Tessera cannot yet determine whether the content is manipulative, fraudulent, or otherwise unacceptable.

This is an intentional honesty boundary. Content-aware policy layers, policy-as-code, and richer contextual evaluation may become future extensions, but they are not current truth.

# 9. Runtime-Native Architecture

Tessera starts as a runtime-native control layer for agent actions.

That means the first credible wedge is not a universal global identity fabric. It is integration into real runtimes where sensitive actions are already happening.

## 9.1 Why runtime-native matters

Runtime-native enforcement has three practical advantages:

1. it sits where the action actually executes,
2. it can fail closed at the boundary,
3. and it can produce decision evidence tied to the runtime context.

## 9.2 OpenClaw as the first wedge

OpenClaw is a strong example of this wedge because it exposes live runtime hook boundaries. Tessera can attach to those boundaries and make real allow or deny decisions on concrete actions.

Current enforcement is still coarse, but it is real. The present wedge already supports runtime-native control over action classes such as:

- `exec.shell`,
- `code.write`,
- and the runtime hook boundary for `message.send`.

This is important product discipline. Tessera should not claim broad authority coverage where only advisory or proxy-level control exists.

## 9.3 Richer runtime context improves policy quality

Many runtimes today expose only limited action context, often just a tool name and parameters. That is enough to build a credible first wedge, but richer context will improve:

- audit quality,
- approval flows,
- contextual policy decisions,
- and operator confidence.

## 10. Revocation

Revocation is one of Tessera's strongest practical advantages.

If an operator decides an agent should stop acting, that authority should not linger until the next login rotation, human approval flow, or deployment cycle. It should stop immediately enough to function as a true kill switch.

This is why Tessera treats revocation as part of the live authority model rather than a background administrative concern.

In practice, Tessera revocation should support:

- explicit invalidation of a delegation instance,
- one-to-one revocation identity,
- fast verifier checks,
- and clear operator visibility that authority has been shut down.

The implementation may vary by deployment, but the product requirement is fixed:

> revocation must be fast enough to shut down agent authority in practice, not just in theory.

## 11. Auditability and Decision Trace

Enforcement alone is not enough. Operators also need to understand what happened.

Tessera therefore treats auditability as a first-class feature, not a secondary log sink.

A useful decision trace should answer:

- what action was requested,
- what capability or credential was presented,
- what runtime or agent requested it,
- whether the action was allowed or denied,
- why that decision occurred,
- and when authority was granted, revoked, or expired.

This audit timeline is valuable both operationally and commercially. It makes agent authority legible to developers, operators, enterprise reviewers, and eventually platforms that need to classify agents as verified, restricted, or revoked.

# 12. Policy Model and Role Templates

One of the main adoption bottlenecks is policy definition.

Most developers and operators do not want to author raw capability JSON from scratch. Even when a low-level policy model is expressive, it is not the best interface for everyday adoption.

The practical product direction is:

> policies should feel like choosing a role, not authoring a schema.

## 12.1 Role templates

Tessera should increasingly expose role templates and preset capability profiles such as:

- **Researcher / read-only**
- **Developer / local dev**
- **Assistant / communications**
- **Purchaser / bounded spend**
- **Custom / advanced**

Raw policies should remain available, but role-first onboarding is the adoption path most users will actually tolerate.

## 12.2 Safe defaults

Templates are not just convenience. They create safer defaults, clearer expectations, and faster operator understanding. They also make policy review and audit easier because the authority shape is easier to recognise.

# 13. Deployment Model

The deployment strategy should be managed first, federated later.

## 13.1 Hosted Tessera Cloud

The near-term default should be a hosted Tessera control plane. This is the fastest path to adoption, the cleanest operator experience, and the easiest way to ship:

- issuer services,
- revocation checks,
- audit timelines,

- policy management,
- and role-template onboarding.

### 13.2 Self-hosted enterprise Tessera

Enterprises will eventually need self-hosted deployment for policy control, internal trust boundaries, compliance needs, and data locality.

### 13.3 Federated ecosystem deployment

Longer term, Tessera should support multiple issuers, multiple revocation registries, and interoperable verification across organisational boundaries.

That federation does not require a blockchain. It can be built with standard issuer trust, registry distribution, and verifier interoperability.

The important sequencing is adoption first, federation later.

## 14. Go-to-Market and Product Wedge

Tessera should not lead with universal claims. It should lead with the strongest real wedge.

That wedge is:

> runtime-native authority control for agent actions in real runtimes.

The starting value proposition is concrete:

- scoped delegation,
- runtime enforcement,
- fast revocation,
- and audit trails for sensitive agent actions.

This is a more believable adoption story than claiming to already be global infrastructure for the agentic web.

### 14.1 The adoption analogy

In adoption terms, the long-term ambition is closer to HTTPS than to a niche developer add-on: a world where non-use becomes visibly unsafe or unacceptable, and platforms begin to label agents as verified, restricted, or revoked.

In user experience terms, the eventual product could feel closer to a VPN:

- one toggle,

- green means protected,
- and most of the machinery remains invisible.

But the core technical analogy remains capability delegation. Tessera is closer to OAuth for agent actions.

## 15. Roadmap

| Phase | Scope |
|---|---|
| Phase 1 — Runtime enforcement wedge | Runtime-native enforcement in real agent environments. Scoped capabilities, revocation, and decision evidence for concrete action classes. |
| Phase 2 — Operator clarity and policy UX | Audit timeline, clearer authority visibility, decision traces, and role-template onboarding. |
| Phase 3 — Hosted control plane and enterprise deployment | Tessera Cloud, self-hosted enterprise deployment, issuer and revocation services, org controls, and policy administration. |
| Phase 4 — Richer runtime context and policy layers | Better action context, more expressive policies, policy suggestion, and more contextual approval flows. |
| Phase 5 — Optional federation | Multi-issuer interoperability, federated verification, and broader platform adoption if the market warrants it. |

Table 1: Sharpened Tessera roadmap.

## 16. Long-Term Vision

The long-term vision is that Tessera becomes a common authority layer for agent actions.

That does not mean claiming infrastructure status in the present tense. It means pursuing a path where:

- runtimes expose real enforcement hooks,
- operators expect scoped and revocable authority,
- platforms distinguish between verified and unverified agents,
- and non-use of authority controls becomes visibly risky.

If that happens, Tessera or a system like it could become part of the standard security posture

of agent systems.

But that outcome depends on adoption, not ideology. The gating factor is whether Tessera solves concrete operator problems better than ambient authority, repeated prompts, or framework-specific controls.

## 17. Conclusion

The core security problem in agent systems is ambient authority.

Agents should not operate with broad, implicit, unrevocable permissions. Sensitive actions should require explicit capabilities that are scoped, enforced at runtime, revocable, and auditable.

That is the central claim of Tessera.

Today, Tessera starts with a narrow but credible wedge: runtime-native enforcement in real agent runtimes. Over time, it can expand into a broader authority layer for agents, but only if it remains grounded in real execution boundaries, honest product limits, and practical operator workflows.

The right quality bar is not grandeur. It is whether a technically literate reader can say:

> this is narrower than I expected, but much more real.

## 18. Get Involved

Tessera is looking for:

- runtime integration partners,
- security reviewers,
- design partners for operator policy UX,
- and contributors to runtime-native guard integrations.

GitHub: https://github.com/tessera-protocol/tessera

Tessera Working Draft v0.7 — April 2026 — Licensed under CC BY 4.0